

Week 12

1. Java Networking

Dr. Andreas S. Maniatis
Assistant Professor

School of Computer Science
COMP.2800 – Software Development [2026W]

Monday, March 23rd, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

● Agenda

- • About Networks
- Java Networking
- Wrap-up



Part I

Networks

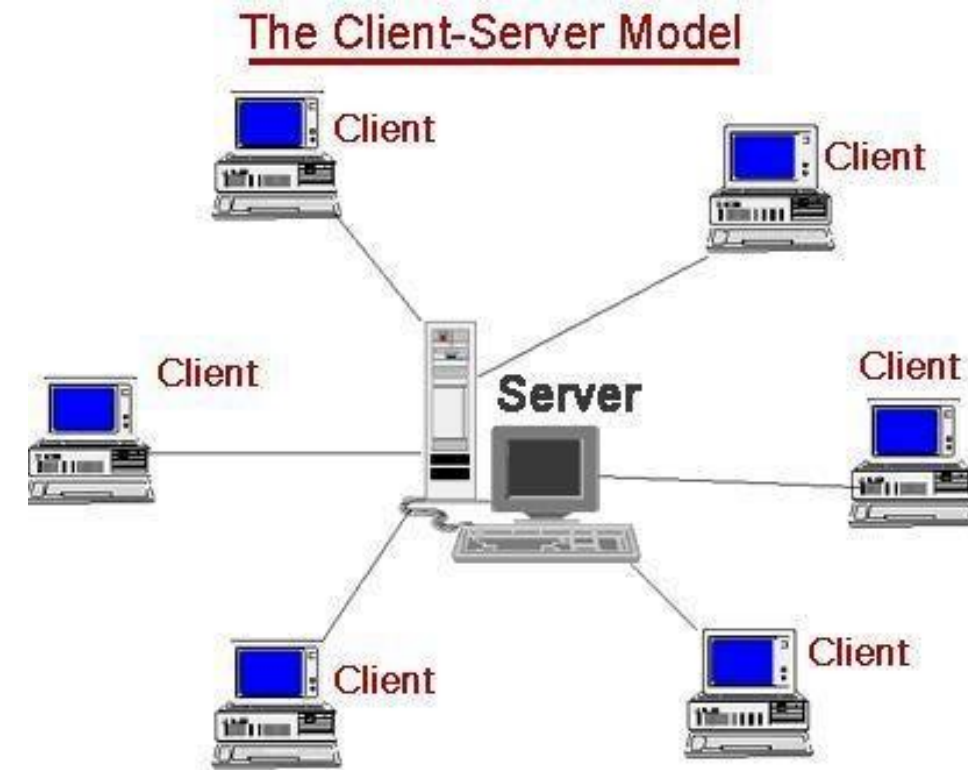


What is a Network?



What is a Network?

- A network is a collection of multiple clients that are connected to (typically) one server.
- If any client wants to communicate with another, they are going to first send the data to the server and then the server can direct that data to the final destination.
- The router (not shown) allows the travel of this data between clients and server. Without some sort of router, this would not be possible.

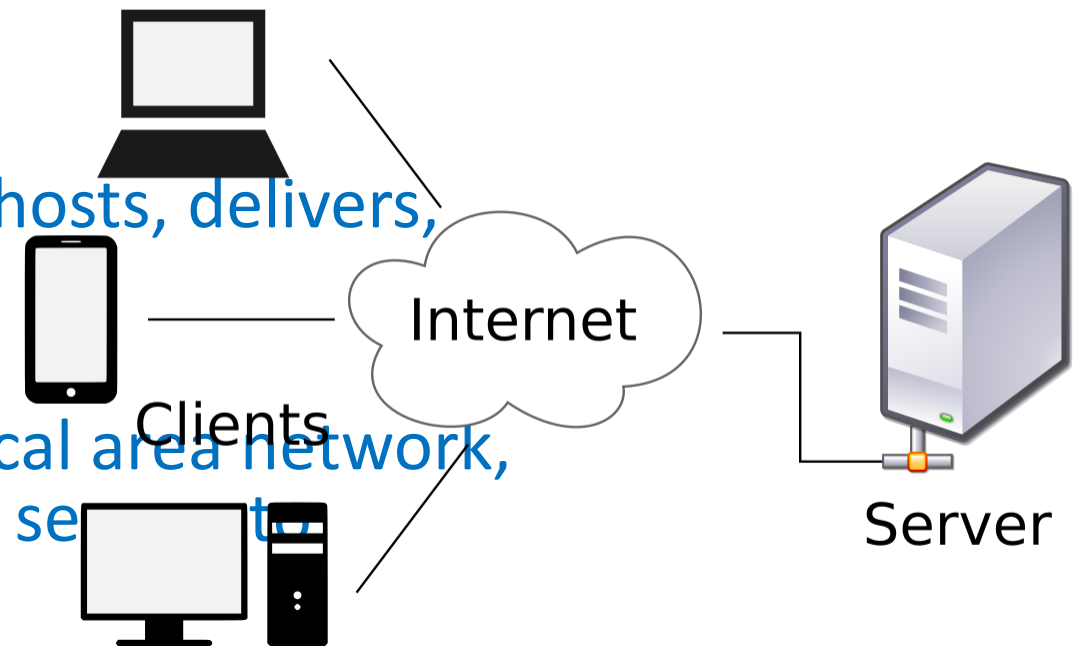


○ Client/Server Architecture



Client/Server Architecture

- **Client:**
A device or application (e.g., web browser, mobile app, desktop application) that initiates requests for resources or services.
- **Server:**
A powerful computer or system that hosts, delivers, and manages resources and services.
- **Network:**
The communication medium (e.g., local area network, the internet) that enables clients and servers to interact and exchange data.



Client/Server...

Client/Server

The client/server model is the standard model for network applications. A typical scenario is as follows:

- The server process is started on some computer. It initializes itself, then goes to sleep waiting to be contacted by a client process for some service.
- A client process is started on some computer connected to the server's computer with a network. It sends a request across the network to the server asking for some service, remote login for instance.
- The server provides its service in two ways:
 1. For a short amount of time service, the server process handle the service itself and goes back to sleep, waiting for new requests, otherwise
 2. The server invokes another process to handle each client request, so that the server process can go back to sleep.

Part II

IP Addresses: Ports → Sockets



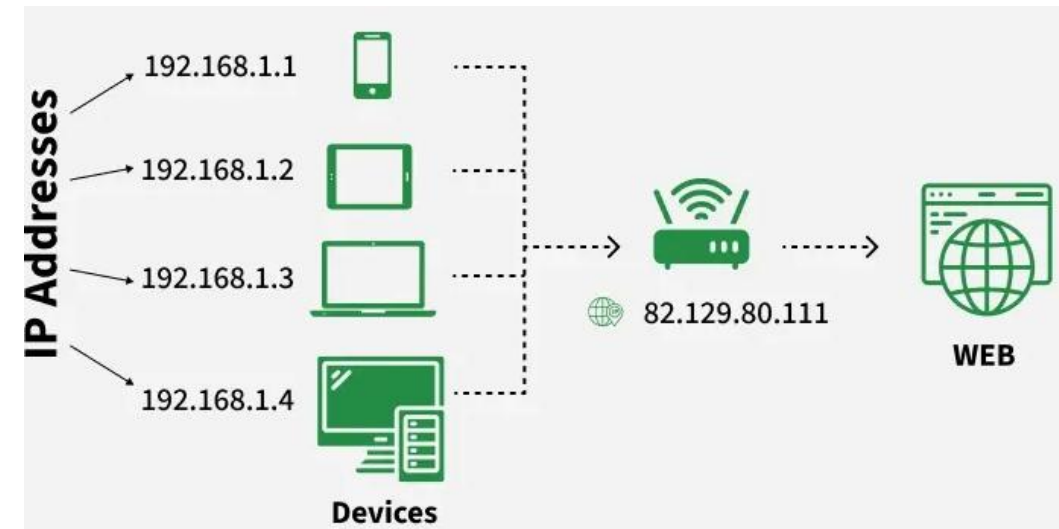
University of Windsor

IP Addresses



IP Addresses

- A unique numerical label assigned to devices connected to a computer network for communication, like a mailing address.
- Serve two main functions:
 - Identifying a specific device, and
 - Directing data to it.
- There are two main versions:
 - IPv4, and
 - IPv6,and various types, such as public, private, static, and dynamic.





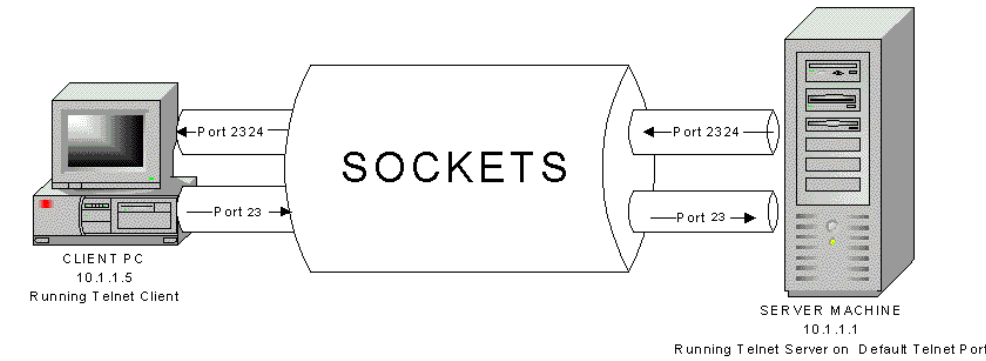
Ports



University of Windsor

Ports

- Logical constructs that identify a specific process or network service on a device.
- Usually a 16-bit number ranging from 1 to 65,535.
- Helps sort incoming traffic to the correct application, preventing a single device from being overwhelmed by multiple services.
- Certain ports are standardized for specific services:
 - Port 80: HTTP (web browsing)
 - Port 443: HTTPS (secure web browsing)
 - Port 25: SMTP (sending email)
 - Port 22: SSH (secure remote access)
- How they work together:
 - An IP address and a port number together form a complete communication endpoint, sometimes called a **socket**.
 - The IP address gets the data to the right computer, and the port number directs it to the correct application on that computer.
- In a URL, the port is often separated from the IP address by a colon, such as in 192.168.1.10:8080.

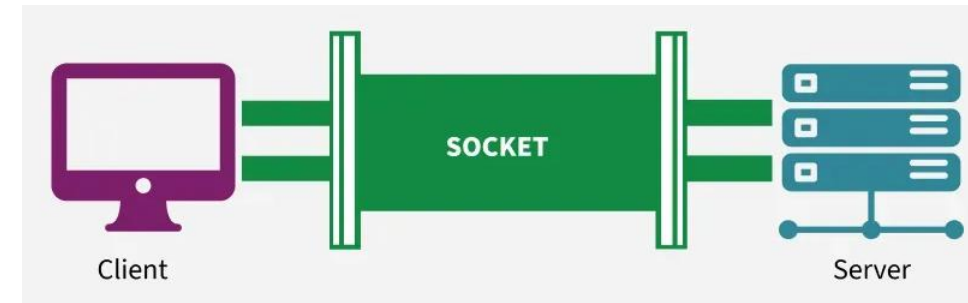




Sockets



IP-Address:Port → Socket



- **IP Address:** A unique numerical label assigned to devices connected to a computer network for communication, like a mailing address. Logical constructs that identify a specific process or network service on a device.
- **Port:**
 - Usually a 16-bit number ranging from 1 to 65,535.
 - Helps sort incoming traffic to the correct application, preventing a single device from being overwhelmed by multiple services.
- **How they work together:**
 - An IP address and a port number together form a complete communication endpoint, sometimes called a **socket**.
 - The IP address gets the data to the right computer, and the port number directs it to the correct application on that computer.

What are Network Sockets?



Each device on the network is identified by an IP address. So, if a client wants to make a connection to the server machine, it is going to need its IP address.



A port number uniquely identifies which application on the machine is the server. A machine could host more than one server as well which share the same IP Address. But a port number will uniquely identify our target server.



A socket can be thought of as a pair of these 2 components. So if you want to create a server or client application, you will need to use a socket. One machine can have multiple sockets, one of which maybe the server we are trying to connect to.



IP Address : Street name



Port Number: Street address

Introducing Sockets

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`

Binding a name to a socket: `bind()`

Listening for connections on a socket: `listen()`

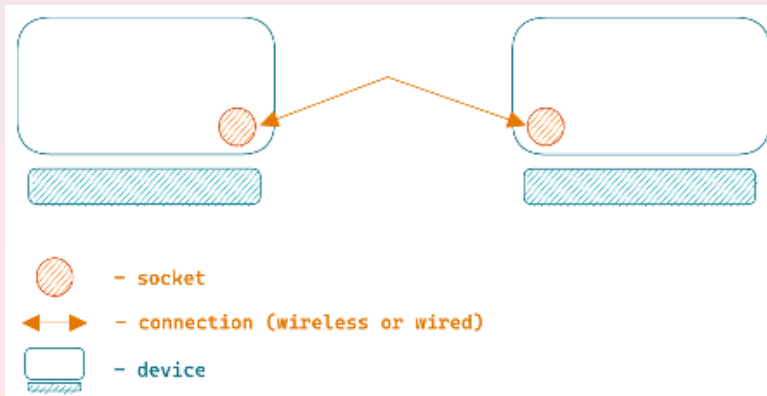
Accepting a connection on a socket: `accept()`

Sockets I

- Sockets are the traditional UNIX IPC mechanism that allows **local/distant** processes to talk to each other.
- IPC using sockets is based on the **client/server** paradigm.

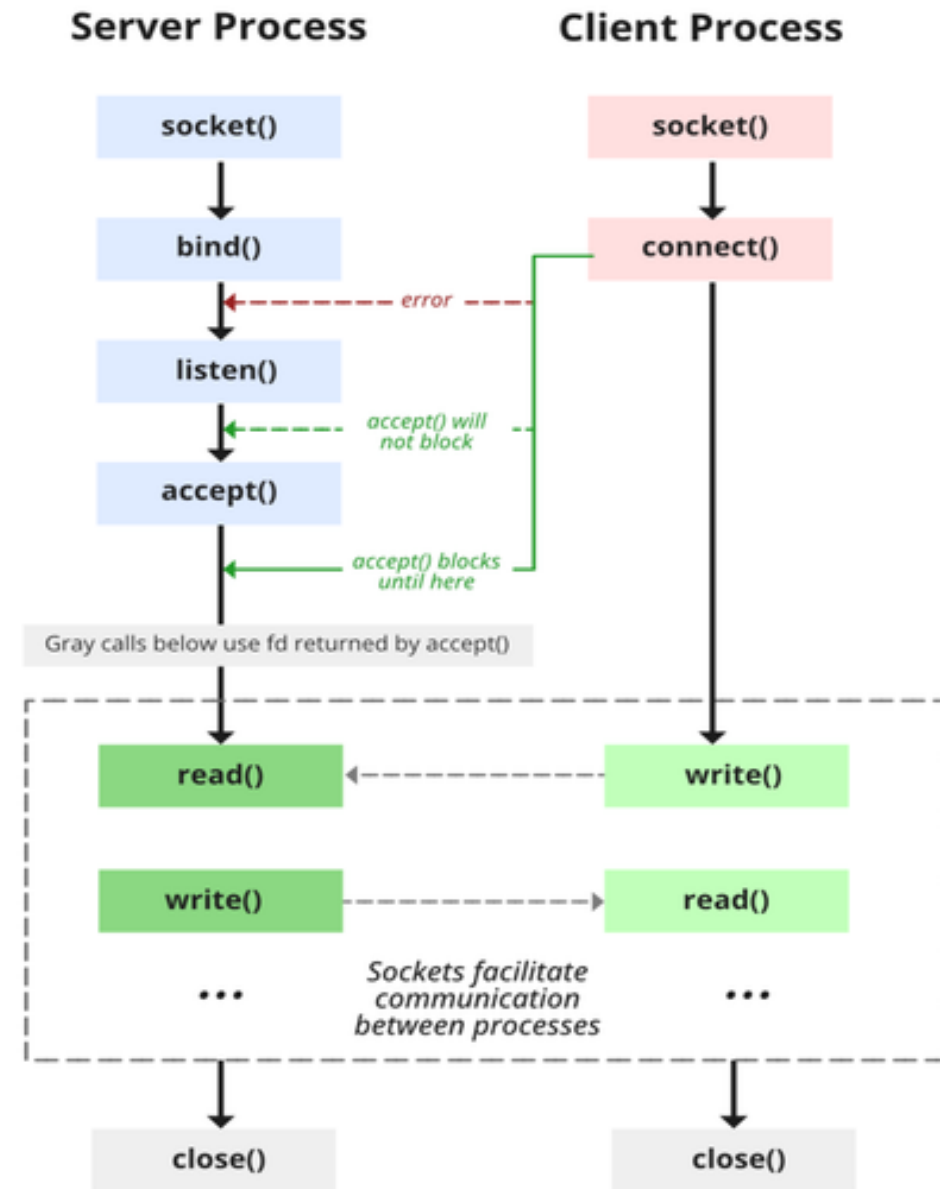
Note

Socket connections are **bidirectional**.



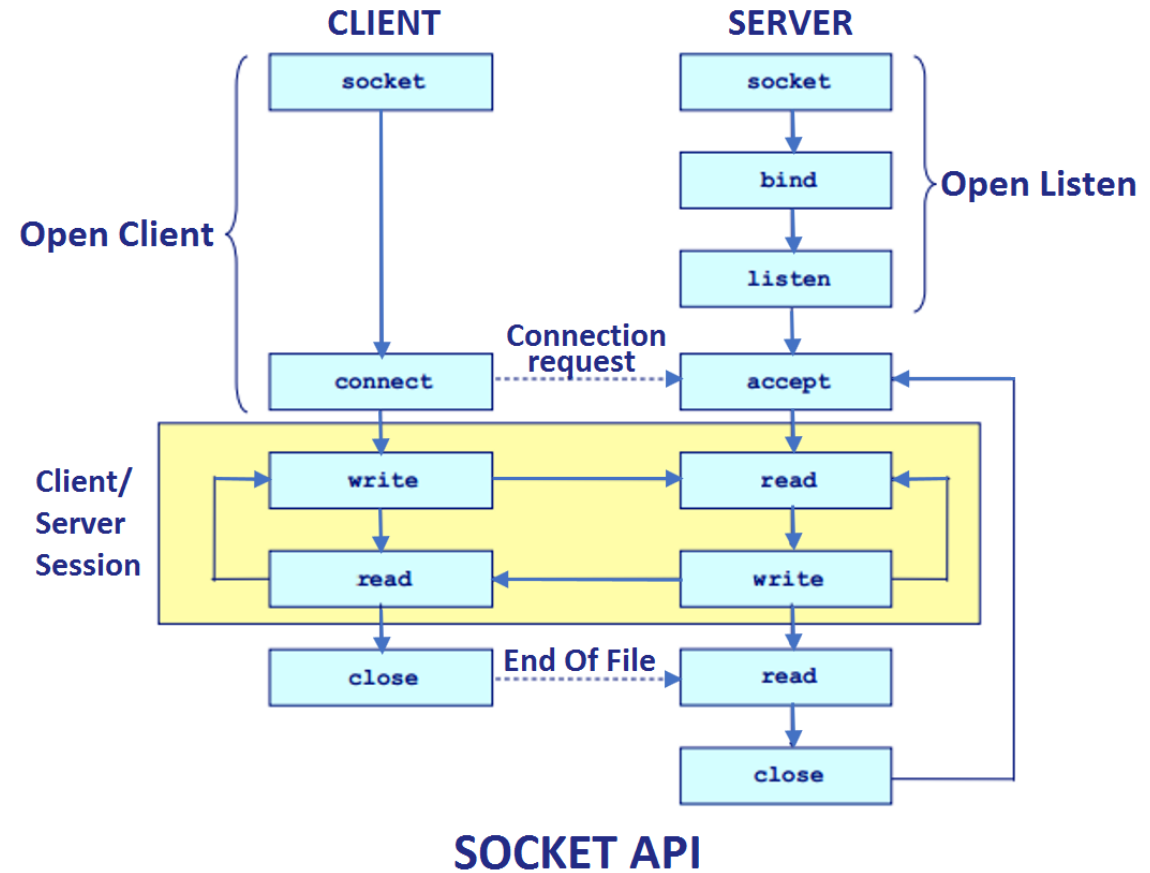
Socket Flow: Review (UNIX & C)

- Server:
 - Create Socket
 - `setsockopt`
 - Bind
 - Listen
 - Accept
 - Send/Receive
 - Close
- Client:
 - Create Socket
 - Connect
 - Send/Receive
 - Close
- Common Issues:
 - Connection failure
 - Port binding errors (already in use)
 - Blocking sockets (default)



Socket Flow: Review (Java)

- Server:
 - Create Socket
 - `setsockopt`
 - Bind
 - Listen
 - Accept
 - Send/Receive
 - Close
- Client:
 - Create Socket
 - Connect
 - Send/Receive
 - Close
- Common Issues:
 - Connection failure
 - Port binding errors (already in use)
 - Blocking sockets (default)





UDP vs TCP

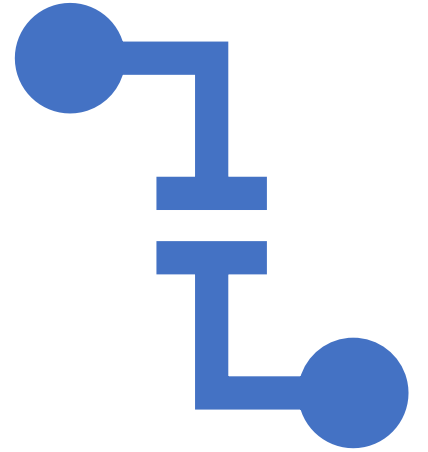




UDP vs TCP



- UDP:
 - Faster, Connectionless, Lossy, Packet-based
- TCP:
 - Slower, Connection-oriented, Guaranteed delivery, Stream-based



The User Datagram Protocol (UDP)

UDP



Use Case :- Video Conferencing



User Datagram Protocol (UDP)

- This type of protocol is used in real-time multiplayer games and video call apps which require fast speeds and can tolerate a little bit of data loss.
- In Java we use the `java.net` package's `DatagramSocket` class to make a socket to “listen” for incoming UDP data.
- There is no concept of a “server” and “client” in UDP, because it's a connectionless protocol. So, these concepts must be manually created by the developer.



● Transmission Control Protocol (TCP)

- • This type of protocol is used in email systems, web browsing, file transfer etc.
- These types of things require reliability and guarantee of data delivery which TCP provides. But it comes with the price of latency.
- UDP can't provide the guarantee of data deliver but it has incredibly low latency since no mechanism is there to ensure data delivery.



UDP vs TCP

Transmission Control Protocol	User Datagram Protocol
A connection-oriented protocol	A connectionless protocol
The most widely used protocol on the internet	Used for voice over IP, streaming video, gaming and live broadcasts
Guarantees that no packets are missing and all the data that's sent makes it to the intended recipient	Faster and needs fewer resources
Sends packets in order so they can be stitched back together easily	Packets don't necessarily arrive in order
Slower and requires more resources	Allows missing packets, but the sender is unable to know whether a packet has been received
Has a bigger header than UDP, best suited for apps that need high reliability and transmission time is relatively less critical	Better suited for applications that need fast, efficient transmission, such as games



● Java's way of handling protocols

- • Java provides a class called `DatagramSocket` for representing a socket that will use the UDP protocol.
- `DatagramPacket` for representing a packet of bytes sent from/received by the socket.
- `ServerSocket` for a TCP socket for the server side. And `Socket` for client side.
The data received and sent by these are represented by streams.





Part II

Java Networking



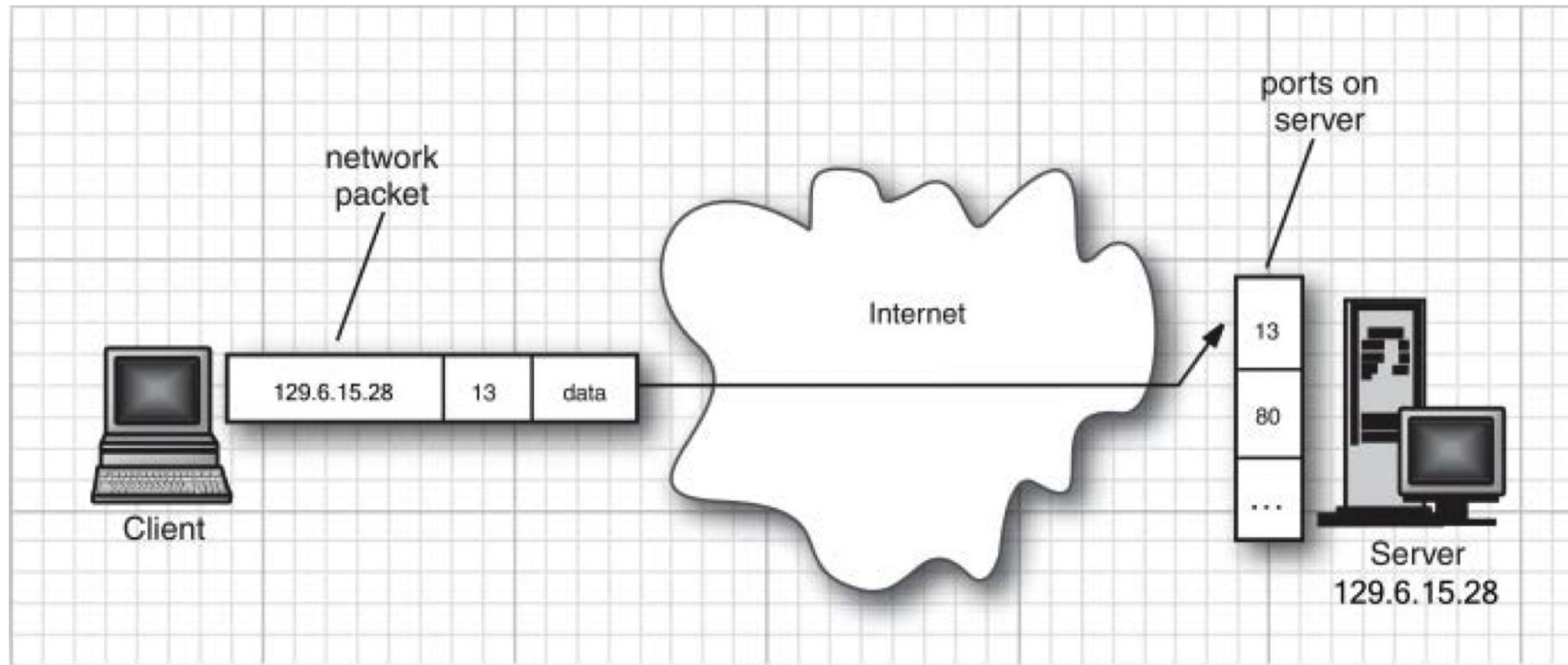
University of Windsor

Java Networking



Connecting to a server

- A port is not a physical device, but an abstraction facilitating communication between a server and a client.



Connecting to a server i

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintStream;
import java.net.Socket;

public class SimpleSocketClientExample
{
    public static void main( String[] args )
    {
        String server = "www.google.com";
        String path = "/";

        System.out.println( "Loading contents of URL: " + server );

        try
        {
            // Connect to the server
            Socket socket = new Socket( server, 80 );

            // Create input and output streams to read from and write to the server
            PrintStream out = new PrintStream( socket.getOutputStream() );
            BufferedReader in = new BufferedReader( new InputStreamReader( socket.getInputStream() ) );

            // Follow the HTTP protocol of GET <path> HTTP/1.0 followed by an empty line
            out.println( "GET " + path + " HTTP/1.0" );
            out.println();

            // Read data from the server until we finish reading the document
            String line = in.readLine();
            while( line != null )
            {
                System.out.println( line );
                line = in.readLine();
            }

            // Close our streams
            in.close();
            out.close();
            socket.close();
        }
        catch( Exception e )
        {
            e.printStackTrace();
        }
    }
}
```



Connecting to a server in Java

java.net.Socket 1.0

- `Socket(String host, int port)`
constructs a socket to connect to the given host and port.
- `InputStream getInputStream()`
- `OutputStream getOutputStream()`
gets the stream to read data from the socket or write data to the socket.

TCP vs UDP

- The examples cover only the Transmission Control Protocol (TCP).
- The Java platform also supports the User Datagram Protocol (UDP), which can be used to send packets (also called datagrams).
- Packets need not be delivered in sequential order to the receiving application and can even be dropped altogether. It is up to the recipient to put the packets in order and to request retransmission of missing packets.
- UDP is well suited for applications in which missing packets can be tolerated—for example, for audio or video streams or continuous measurements.



Socket Timeout

- Reading data from a socket blocks until data are available.
- If your host is not available, you need to wait for a long time.

```
Socket s = new Socket(. . .);  
s.setSoTimeout(10000); // time out after 10 seconds
```

● Socket Timeout

- • The constructor `Socket(String host, int port)` can block indefinitely until an initial connection to the host is established.
- How can you overcome the problem?



Socket Timeout

- The constructor `Socket(String host, int port)` can block indefinitely until an initial connection to the host is established.
- How can you overcome the problem?
`Socket s = new Socket();`
`s.connect(new InetSocketAddress(host, port), timeout);`



Important Methods in `java.net.Socket`

`java.net.Socket` 1.0

- `Socket()` 1.1
creates a socket that has not yet been connected.
- `void connect(SocketAddress address)` 1.4
connects this socket to the given address.
- `void connect(SocketAddress address, int timeoutInMilliseconds)` 1.4
connects this socket to the given address, or returns if the time interval expired.
- `void setSoTimeout(int timeoutInMilliseconds)` 1.1
sets the blocking time for read requests on this socket. If the timeout is reached, an `InterruptedException` is raised.
- `boolean isConnected()` 1.4
returns true if the socket is connected.
- `boolean isClosed()` 1.4
returns true if the socket is closed.



Internet Addresses

- You need to use `InetAddress` class if you need to convert between host names and Internet addresses.

java.net.InetAddress 1.0

- `static InetAddress getByName(String host)`
constructs an `InetAddress`, or an array of all Internet addresses, for the given host name.
- `static InetAddress[] getAllByName(String host)`
constructs an `InetAddress` for the local host.
- `byte[] getAddress()`
returns an array of bytes that contains the numerical address.
- `String.getHostAddress()`
returns a string with decimal numbers, separated by periods, for example "129.6.15.28".
- `String.getHostName()`
returns the host name.



Implementing Servers

- A server program, when started, waits for a client to attach to its port.

Every server program, such as an HTTP web server, continues performing this loop:

1. It receives a command from the client (“get me this information”) through an incoming data stream.
2. It decodes the client command.
3. It gathers the information that the client requested.
4. It sends the information to the client through the outgoing data stream.



Implementing Servers

`java.net.ServerSocket` 1.0

- `ServerSocket(int port)`
creates a server socket that monitors a port.
- `Socket accept()`
waits for a connection. This method blocks (i.e., idles) the current thread until the connection is made. The method returns a `Socket` object through which the program can communicate with the connecting client.
- `void close()`
closes the server socket.

● Implementing Servers

- • What is the problem of the previous example?



● Implementing Servers

- • What is the problem of the previous example?
- See the code examples discussed in the class



Part III

Wrap-up – What's Next



Summary



Summary

In this Hands-on Tutorial lecture:

- We were introduced to Networking:
 - Networks
 - Client / Server Architecture
 - Addresses – Ports – Sockets
- Performed in-class exercises.
 - Server
 - Client
 - Live connection



Next Lecture



University of Windsor

Next Week Lectures

- Last lecture for the semester:
 - Java Microservices
 - Final Exam: Overview
 - Group Project Status
 - How to present successfully
 - Student Perception of Teaching (SPT)
 - Job Opportunities



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor